

Scheduling Algorithm

1.Input:

```
sushmitha@LAPTOP-B9O7BO × + v
GNU nano 8.7.1 fcfs.c
#include <stdio.h>
int main() {
int n, i;
int bt[20], wt[20], tat[20];
printf("Enter number of processes: ");
scanf("%d", &n);
printf("Enter burst time for each process:\n");
for (i = 0; i < n; i++) {
printf("P%d: ", i + 1);
scanf("%d", &bt[i]);
}
// Waiting time
wt[0] = 0;
for (i = 1; i < n; i++) {
wt[i] = wt[i - 1] + bt[i - 1];
}
// Turnaround time
for (i = 0; i < n; i++) {
tat[i] = wt[i] + bt[i];
}
printf("\nProcess\tBT\tWT\tTAT\n");
for (i = 0; i < n; i++) {
printf("P%d\t%d\t%d\t%d\n",
i + 1, bt[i], wt[i], tat[i]);
}
return 0;
}
```

output:

```
sushmitha@LAPTOP-B9O7BO6L:~$ nano fcfs.c
sushmitha@LAPTOP-B9O7BO6L:~$ gcc fcfs.c -o fcfs
sushmitha@LAPTOP-B9O7BO6L:~$ ./fcfs
Enter number of processes: 1
Enter burst time for each process:
P1: 2

Process BT      WT      TAT
P1      2        0        2
```

2.Input:

```

#include <stdio.h>

int main() {
    int n, i, j;
    int bt[20], wt[20], tat[20];
    int temp;

    printf("Enter number of processes: ");
    scanf("%d", &n);

    printf("Enter burst time:\n");

    for (i = 0; i < n; i++) {
        printf("P%d: ", i + 1);
        scanf("%d", &bt[i]);
    }

    // Sort burst times
    for (i = 0; i < n - 1; i++) {
        for (j = i + 1; j < n; j++) {
            if (bt[i] > bt[j]) {
                temp = bt[i];
                bt[i] = bt[j];
                bt[j] = temp;
            }
        }
    }

    // Waiting time
    wt[0] = 0;

    for (i = 1; i < n; i++) {
        wt[i] = wt[i - 1] + bt[i - 1];
    }

    // Turnaround time
    for (i = 0; i < n; i++) {
        tat[i] = wt[i] + bt[i];
    }

    printf("\nProcess\tBT\tWT\tTAT\n");

    for (i = 0; i < n; i++) {
        printf("P%d\t%d\t%d\t%d\n",
            i + 1, bt[i], wt[i], tat[i]);
    }

    return 0;
}

```

Output:

```

sushmitha@LAPTOP-B907B06L:~$ nano sjf_NONpreemptive.c
sushmitha@LAPTOP-B907B06L:~$ gcc sjf_NONpreemptive.c -o sjf_NONpreemptive
sushmitha@LAPTOP-B907B06L:~$ ./sjf_NONpreemptive
Enter number of processes: 4
Enter burst time:
P1: 10
P2: 20
P3: 30
P4: 40

Process BT      WT      TAT
P1      10      0       10
P2      20      10      30
P3      30      30      60
P4      40      60     100
sushmitha@LAPTOP-B907B06L:~$

```

3.Input:

```

GNU nano 8.7.1 priority.c

int main() {
int n, i, j;
int bt[20], priority[20], wt[20], tat[20];
int temp;
printf("Enter number of processes: ");
scanf("%d", &n);
printf("Enter burst time and priority:\n");
for (i = 0; i < n; i++) {
printf("P%d Burst Time: ", i + 1);
scanf("%d", &priority[i]);
}
// Sort according to priority
for (i = 0; i < n - 1; i++) {
for (j = i + 1; j < n; j++) {
if (priority[i] > priority[j]) {
temp = priority[i];
priority[i] = priority[j];
priority[j] = temp;
temp = bt[i];
bt[i] = bt[j];
bt[j] = temp;
}
}
}
// Waiting time
wt[0] = 0;
for (i = 1; i < n; i++) {
wt[i] = wt[i - 1] + bt[i - 1];
}
// Turnaround time
for (i = 0; i < n; i++) {
tat[i] = wt[i] + bt[i];
}
printf("\nProcess\tBT\tPriority\tWT\tTAT\n");
for (i = 0; i < n; i++) {
printf("P%d\t%d\t%d\t\t%d\t%d\n", i + 1, bt[i], priority[i], wt[i], tat[i]);
}
}

```

```
return 0;  
}
```

Output:

```
sushmitha@LAPTOP-B907B06L:~$ nano priority.c  
sushmitha@LAPTOP-B907B06L:~$ nano priority.c  
sushmitha@LAPTOP-B907B06L:~$ gcc priority.c -o priority  
sushmitha@LAPTOP-B907B06L:~$ ./priority
```

Enter number of processes: 3

Enter burst time and priority:

P1 Burst Time: 3

P2 Burst Time: 3

P3 Burst Time: 30

Process	BT	Priority	WT	TAT
P1	0	3	0	0
P2	0	3	0	0
P3	0	30	0	0

4.Input:



sushmitha@LAPTOP-B9O7BO



GNU nano 8.7.1

#include <stdio.h>

```
int main() {
    int n, i;
    int bt[20], rem_bt[20];
    int wt[20], tat[20];
    int quantum;
    int time = 0;
    int done;

    printf("Enter number of processes: ");
    scanf("%d", &n);

    printf("Enter burst time:\n");

    for (i = 0; i < n; i++) {
        printf("P%d: ", i + 1);
        scanf("%d", &bt[i]);

        rem_bt[i] = bt[i];
        wt[i] = 0;
    }

    printf("Enter time quantum: ");
    scanf("%d", &quantum);

    // Round Robin
    while (1) {
        done = 1;

        for (i = 0; i < n; i++) {
            if (rem_bt[i] > 0) {
                done = 0;
            }
        }
    }
}
```

```

        if (rem_bt[i] > quantum) {
            time = time + quantum;
            rem_bt[i] = rem_bt[i] - quantum;
        }
        else {
            time = time + rem_bt[i];

            wt[i] = time - bt[i];

            rem_bt[i] = 0;
        }
    }

    if (done == 1)
        break;
}

// Turnaround time
for (i = 0; i < n; i++) {
    tat[i] = wt[i] + bt[i];
}

printf("\nProcess\tBT\tWT\tTAT\n");

for (i = 0; i < n; i++) {
    printf("P%d\t%d\t%d\t%d\n",
           i + 1, bt[i], wt[i], tat[i]);
}

return 0;
}

```

Output:

```
sushmitha@LAPTOP-B907B06L:~$ nano roundrobin.c
sushmitha@LAPTOP-B907B06L:~$ gcc roundrobin.c -o roundrobin
sushmitha@LAPTOP-B907B06L:~$ ./roundrobin
Enter number of processes: 3
Enter burst time:
P1: 10
P2: 20
P3: 30
Enter time quantum: 3

Process BT      WT      TAT
P1      10      18      28
P2      20      28      48
P3      30      30      60
sushmitha@LAPTOP-B907B06L:~$ |
```